Universite de Rouen Normandie

Master 1 – Genie Informatique et Logiciel

Projet Gestion de Flotte de vehicules

Rapport de Semaine 3

Microservice Véhicules

Etudiant :
Ahcene Amouchas
Florian Pepin

Encadrants :
Lydia & Luc

Annee universitaire :
2025 – 2026

Semaine :
3 / 7

25 mars 2026

Table des matières

1 Introduction et objectifs de la semaine **2**

2 Mise en œuvre du microservice de gestion des véhicules **3**
2.1 Modèle de données et Soft Delete . . . 3
2.2 Couche Accès aux Données (Repository) . . . 3
2.3 Logique Métier et DTOs . . . 4
2.4 Contrôleur REST . . . 4

3 Tests de l'API REST avec Postman **4**
3.1 CRUD Véhicules . . . 5
3.2 Gestion des assignations . . . 9
3.3 Bilan des tests . . . 10

4 Résolveurs GraphQL **10**
4.1 Architecture des résolveurs . . . 10
4.2 Résolveurs Véhicules . . . 11
4.3 Cohérence REST / GraphQL . . . 12
4.4 Gestion des stubs . . . 12

1. Introduction et objectifs de la semaine

Objectif de la semaine 3 : Développer le premier microservice fonctionnel du service véhicules en respectant les bonnes pratiques de qualité logicielle : tests, couverture de code, et observabilité intégrée dès le départ.

La semaine 3 du projet marque le passage de l'infrastructure au développement métier. L'équipe s'est concentrée sur la conception et l'implémentation du service de véhicules qui est le socle du système de gestion de flotte.

Les grandes étapes réalisées sont les suivantes :

1. Développement de l'**API REST** complète (CRUD véhicules avec validation des données)
2. Intégration d'**Apache Kafka** en tant que producer/consumer pour les événements métier
3. Création des **résolveurs GraphQL** pour l'agrégation future entre services
4. **Tests unitaires (>80% couverture) et d'intégration**
5. Instrumentation du service avec le **SDK OpenTelemetry** (traces, métriques, logs structurés)

2. Mise en œuvre du microservice de gestion des véhicules

L'une des briques centrales du système est le microservice de gestion de la flotte. Cette section détaille l'implémentation du CRUD (Create, Read, Update, Delete) pour l'entité `Vehicle`, en suivant une architecture en couches propre à Spring Boot.

2.1. Modèle de données et Soft Delete

L'entité `Vehicle` est conçue pour être persistée dans une base de données PostgreSQL. Une particularité de l'implémentation est l'usage du **Soft Delete** : au lieu de supprimer physiquement une ligne, nous renseignons un champ `deletedAt`.

```
@Entity
@Table(name = "vehicles")
public class Vehicle {
    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;

    @Column(name = "plate_number", nullable = false, unique = true)
    private String plateNumber;

    @Enumerated(EnumType.STRING)
    @JdbcTypeCode(SqlTypes.NAMED_ENUM)
    private VehicleStatus status = VehicleStatus.available;

    private OffsetDateTime deletedAt; // Pour le Soft Delete
}
```

Listing 1 – Entité Vehicle avec énumérations PostgreSQL

2.2. Couche Accès aux Données (Repository)

Pour supporter le mécanisme de suppression logique, le `VehicleRepository` définit des requêtes JPQL spécifiques qui filtrent les enregistrements dont la date de suppression est nulle.

```
@Repository
public interface VehicleRepository extends JpaRepository<Vehicle,
   UUID> {
    @Query("SELECT v FROM Vehicle v WHERE v.deletedAt IS NULL")
    List<Vehicle> findAllActive();

    @Query("SELECT v FROM Vehicle v WHERE v.id = :id AND v.
       deletedAt IS NULL")
    Optional<Vehicle> findByIdActive(UUID id);
}
```

Listing 2 – Requêtes personnalisées pour le Soft Delete

2.3. Logique Métier et DTOs

L'exposition des données ne se fait pas directement via l'entité, mais par des **Java Records** (DTOs), garantissant l'immutabilité et la validation des entrées.

— **Validation** : Utilisation de `@NotBlank` et `@Min` dans `VehicleInput` pour valider les payloads.

— **Mapping** : Le service transforme les entités en `VehicleResponse` avant de les retourner au contrôleur.

2.4. Contrôleur REST

Le `VehicleController` expose les points de terminaison nécessaires à la gestion de la flotte. Voici le récapitulatif des routes implémentées :

Méthode	Point de terminaison	Action
GET	`/vehicles`	Liste tous les véhicules actifs
GET	`/vehicles/{id}`	Détails d'un véhicule spécifique
POST	`/vehicles`	Création d'un véhicule (Validation active)
PUT	`/vehicles/{id}`	Mise à jour des informations
PATCH	`/vehicles/{id}/status`	Modification rapide du statut
DELETE	`/vehicles/{id}`	Suppression logique (Soft Delete)

```
@DeleteMapping("/{id}")
@ResponseStatus(HttpStatus.NO_CONTENT)
public void deleteVehicle(@PathVariable UUID id) {
    service.deleteVehicle(id);
}
```

Listing 3 – Extrait du contrôleur pour la suppression logique

Cette structure assure que les données critiques restent en base pour la traçabilité (historique des livraisons) tout en étant invisibles pour les opérations courantes de l'application.

3. Tests de l'API REST avec Postman

> **Objectif :** Valider le bon fonctionnement de l'ensemble des endpoints du service Véhicules via des tests manuels réalisés avec **Postman**, couvrant les opérations CRUD ainsi que la gestion des assignations.

L'ensemble des routes exposées par le service Véhicules ont été testées manuellement à l'aide de Postman. Les tests couvrent les deux ressources principales : les **véhicules** et leurs **assignations**. Chaque requête a retourné le code HTTP attendu, confirmant la conformité de l'API aux standards REST.

3.1. CRUD Véhicules

POST /vehicles — Ajout d'un véhicule

La création d'un véhicule s'effectue via une requête `POST` sur `/vehicles`. Le corps de la requête contient les informations du véhicule (`plateNumber`, `brand`, `model`, `fuelType`, `mileageKm`, `vin`, `payloadCapacityKg`, `cargoVolumeM3`). Le service retourne un **201 Created** avec l'objet créé, incluant l'UUID généré automatiquement et le statut par défaut `available`.

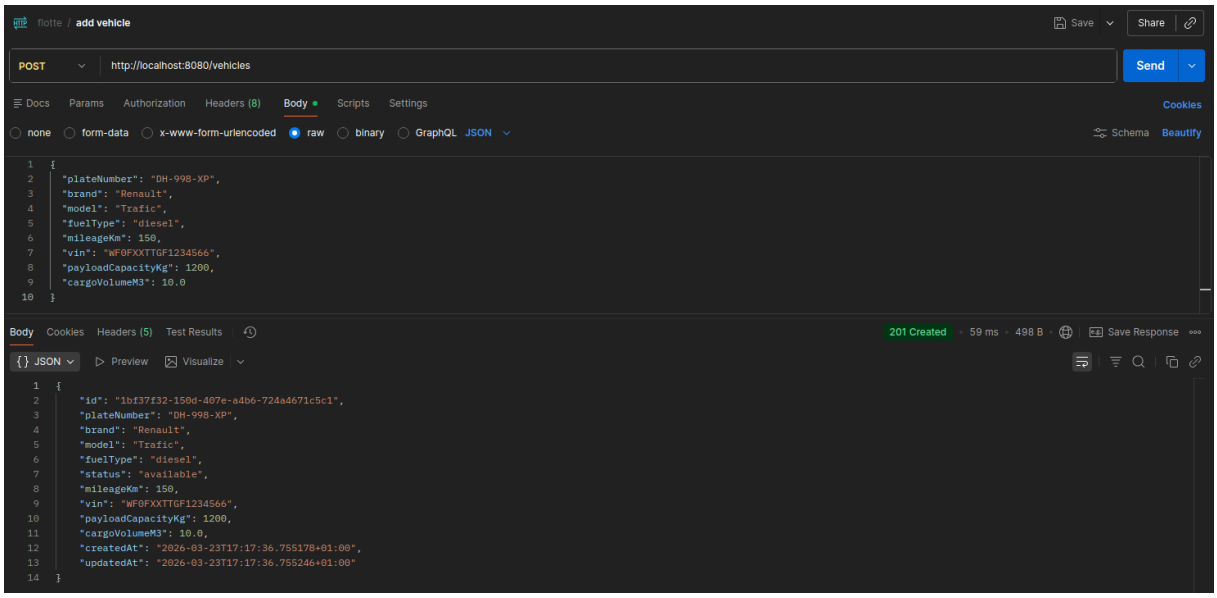

FIGURE 1 – POST /vehicles — Création d'un véhicule (201 Created)

	id [PK] uuid	plate_number character varying (20)	brand character varying (100)	model character varying (100)	fuel_type fuel_type	mileage_km integer	status vehicle_status
1	1bf37f32-150d-407e-a4b6-724a4671c5c1	DH-998-XP	Renault	Trafic	diesel	150	on_delivery
2	277223db-85d2-4543-8542-9c179ed393...	DH-999-XP	Ford	Transit	diesel	200	in_maintenance

FIGURE 2 – POST /vehicles — Détails

GET /vehicles — Liste de tous les véhicules

Une requête `GET` sur `/vehicles` retourne la liste des véhicules non supprimés. Le mécanisme de **soft delete** est ici mis en évidence : seuls les véhicules actifs sont retournés, même si la base de données en contient davantage. Le serveur répond avec un **200 OK**.

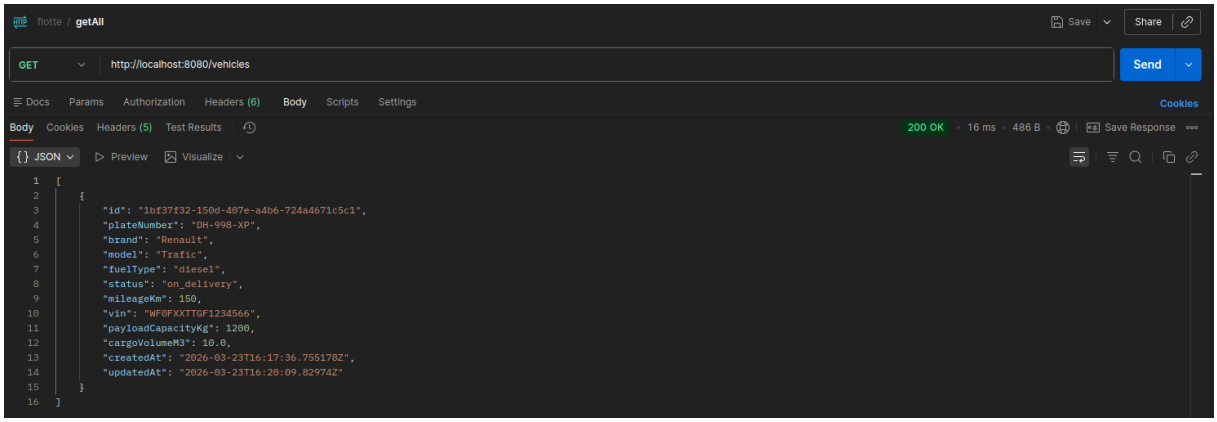

FIGURE 3 – GET /vehicles — Récupération de la liste des véhicules (200 OK)

GET /vehicles/ :id — Récupération par identifiant

La récupération d'un véhicule précis s'effectue en passant son UUID en paramètre de chemin. Le service retourne un **200 OK** avec l'intégralité des champs du véhicule correspondant.

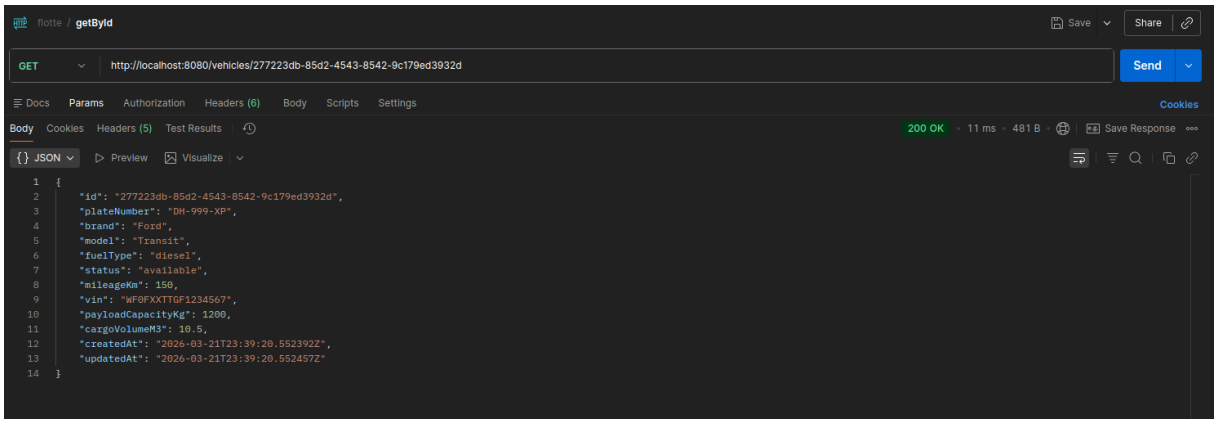

FIGURE 4 – GET /vehicles/ :id — Récupération d'un véhicule par UUID (200 OK)

PUT /vehicles/ :id — Mise à jour complète

La mise à jour d'un véhicule via `PUT` remplace les champs modifiables (`mileageKm`, `brand`, `model`, etc.). Le service retourne un **200 OK** avec les données mises à jour et un `updatedAt` rafraîchi.

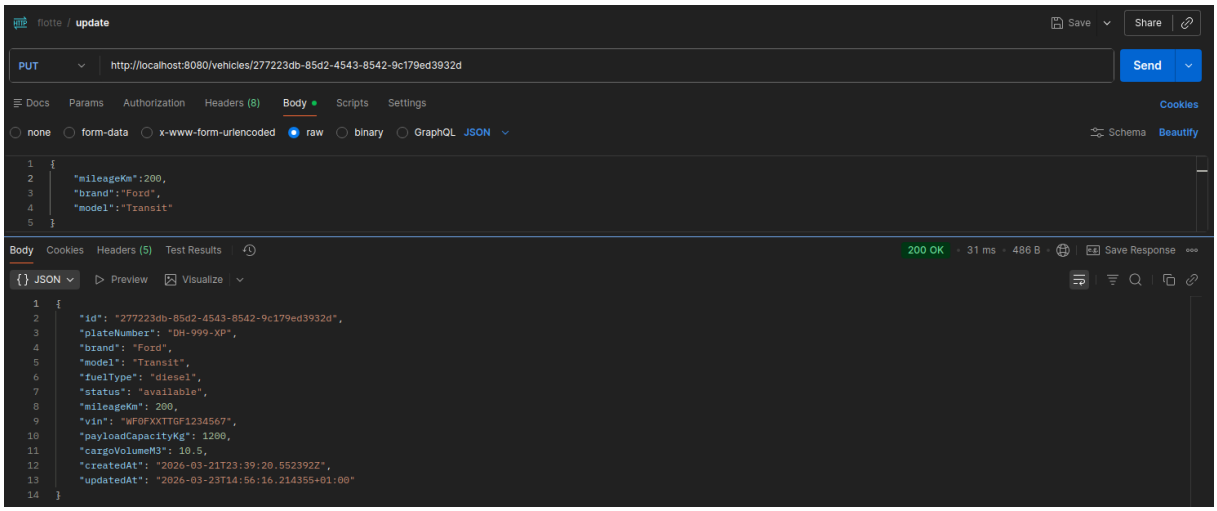

FIGURE 5 – PUT /vehicles/ :id — Mise à jour complète d'un véhicule (200 OK)

PATCH /vehicles/ :id/status — Mise à jour du statut

Une route dédiée permet de modifier uniquement le statut d'un véhicule (ex. `available`, `in_maintenance`, `on_delivery`). Cette approche évite d'envoyer l'intégralité du corps pour un changement ponctuel. Le service répond avec un **200 OK** et retourne le véhicule avec son nouveau statut.

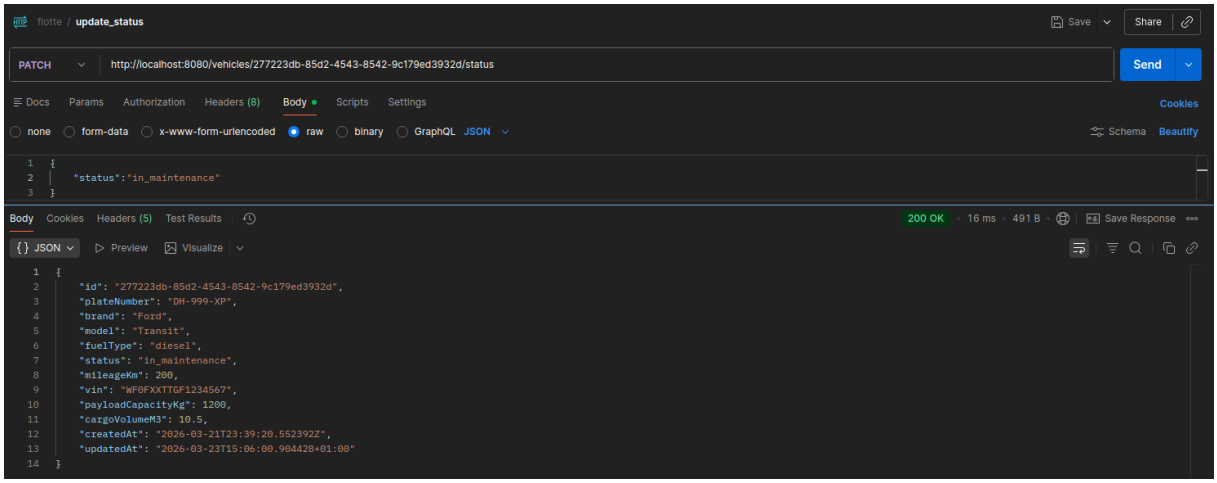

FIGURE 6 – PATCH /vehicles/ :id/status — Mise à jour du statut (200 OK)

DELETE /vehicles/ :id — Suppression logique

La suppression d'un véhicule est implémentée en **soft delete** : le véhicule n'est pas physiquement retiré de la base de données mais marqué comme supprimé. Le service retourne un **204 No Content**, et le véhicule disparaît des résultats des requêtes `GET`. La base de données (visible via pgAdmin) conserve bien les deux entrées, confirmant ce comportement.

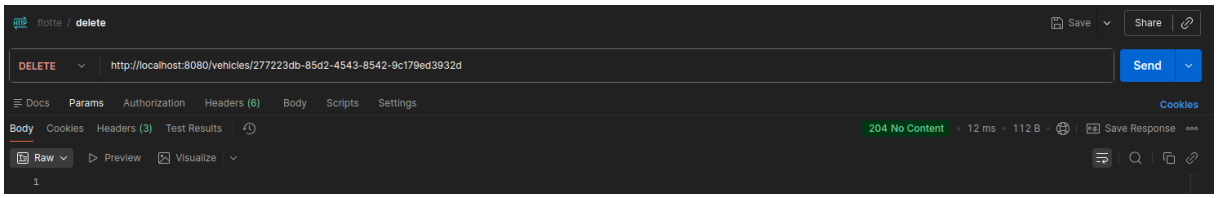

FIGURE 7 – DELETE /vehicles/ :id — Suppression logique (204 No Content)

3.2. Gestion des assignations

POST /vehicles/ :id/assignments — Ajout d'une assignation

L'assignation d'un conducteur à un véhicule s'effectue via une requête `POST` sur `/vehicles/:id/assi`
Le corps contient le `driverId`, des `notes` optionnelles et le `createdBy`. Le service répond avec un **201 Created** et retourne l'assignation créée avec sa date de début.

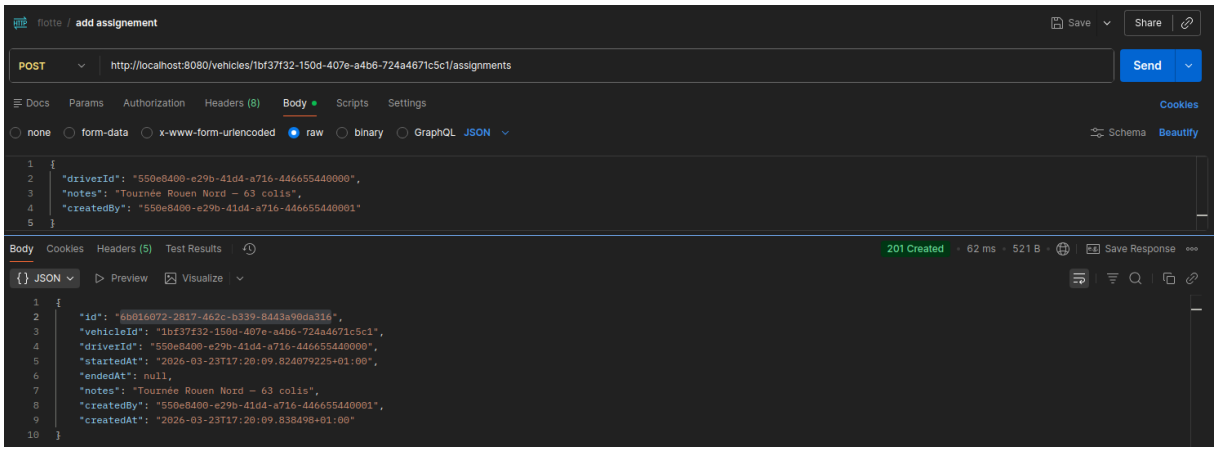

FIGURE 8 – POST /vehicles/ :id/assignments — Création d'une assignation (201 Created)

GET /vehicles/ :id/assignments — Liste des assignations

La récupération des assignations d'un véhicule retourne l'historique complet des affectations de conducteurs. Le champ `endedAt` à `null` indique une assignation en cours. Le service répond avec un **200 OK**.

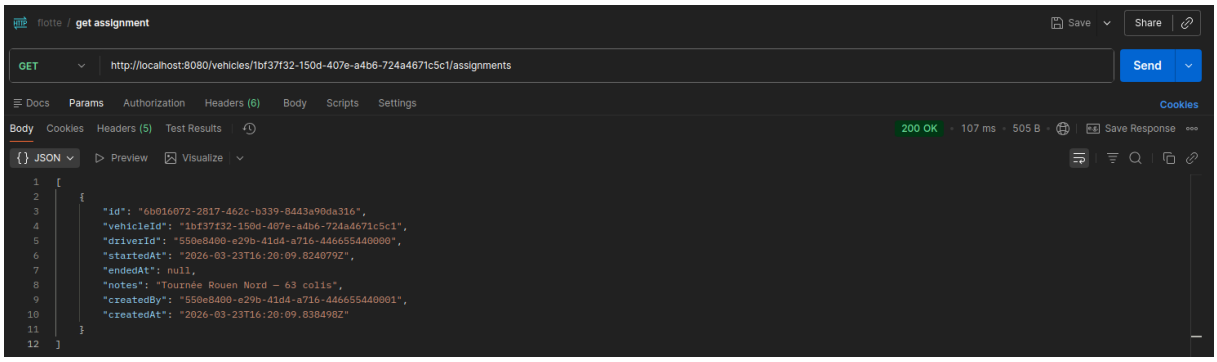

FIGURE 9 – GET /vehicles/ :id/assignments — Liste des assignations (200 OK)

3.3. Bilan des tests

Requête	Endpoint	Code attendu	Résultat
POST	/vehicles	201 Created	**OK**
GET	/vehicles	200 OK	**OK**
GET	/vehicles/ :id	200 OK	**OK**
PUT	/vehicles/ :id	200 OK	**OK**
PATCH	/vehicles/ :id/status	200 OK	**OK**
DELETE	/vehicles/ :id	204 No Content	**OK**
POST	/vehicles/ :id/assignments	201 Created	**OK**
GET	/vehicles/ :id/assignments	200 OK	**OK**

L'ensemble des 8 endpoints testés retournent les codes HTTP attendus. Le soft delete est correctement implémenté et le filtrage des véhicules supprimés fonctionne comme prévu.

4. Résolveurs GraphQL

> **Objectif :** Exposer les données du service Véhicules via une couche **GraphQL** portée par l'API Gateway, en séparant clairement les résolveurs fonctionnels des résolveurs non encore branchés aux microservices.

L'API Gateway agrège les résolveurs de l'ensemble des microservices en un schéma GraphQL unifié. Pour la semaine 3, seuls les résolveurs liés au service Véhicules sont pleinement opérationnels. Les autres (conducteurs, alertes, localisation, maintenance) sont provisoirement remplacés par des **stubs** qui retournent une erreur explicite, évitant tout comportement silencieux.

4.1. Architecture des résolveurs

Les résolveurs sont organisés en trois fichiers distincts :

- `vehicleResolvers.ts` : résolveurs actifs, branchés sur le service Véhicules.
- `stubResolvers.ts` : résolveurs non encore implémentés, retournant une erreur `NOT_IMPLEMENTED`.
- `index.ts` : point d'entrée fusionnant les deux via spread operator.

Le fichier `index.ts` assemble les résolveurs de façon non destructive : les clés de `vehicleResolvers` écrasent celles des stubs pour les champs déjà implémentés, ce qui permet d'enrichir progressivement le schéma semaine après semaine sans modifier la structure d'agrégation.

4.2. Résolveurs Véhicules

Queries

Trois queries sont exposées et délèguent directement au contexte GraphQL (`ctx.vehicle`) :

- `vehicles` : liste des véhicules avec filtrage optionnel par statut (`status`, `limit`, `offset` avec valeurs par défaut de 20 et 0).
- `vehicle` : récupération d'un véhicule par son UUID.
- `vehicleAssignments` : récupération de l'historique des assignations d'un véhicule via son `vehicle_id`.

Mutations

Six mutations sont implémentées :

- `createVehicle` : création d'un véhicule avec ses champs obligatoires (`plate_number`, `brand`, `model`, `fuel_type`, `mileage_km`, `payload_capacity_kg`, `cargo_volume_m3`) et optionnels (`vin`, `color`).
- `updateVehicle` : mise à jour complète des champs modifiables d'un véhicule (`brand`, `model`, `mileage_km`, `vin`, `color`).
- `updateVehicleStatus` : mise à jour ciblée du seul statut d'un véhicule.
- `deleteVehicle` : suppression logique d'un véhicule (soft delete), retourne un booléen confirmant l'opération.
- `assignVehicle` : assignation d'un conducteur à un véhicule via leurs UUIDs.
- `unassignVehicle` : désassignation du conducteur actuel d'un véhicule.

Résolveurs de types

Le type `Vehicle` définit des résolveurs de champs pour les relations non encore implémentées (`current_location`, `current_assignment`, `maintenance_records`), retournant respectivement `null` ou un tableau vide. Cette approche permet au schéma de rester valide sans bloquer les requêtes côté client.

Le type `Assignment` résout ses deux relations :

- `vehicle` : récupère le véhicule lié via `ctx.vehicle.getVehicle()` en validant la présence du `vehicle_id`.
- `driver` : récupère le conducteur lié via `ctx.driver.getDriver()` en validant la présence du `driver_id`.

Dans les deux cas, une `GraphQLError` avec le code `NOT_FOUND` ou `INVALID_UPSTREAM` est levée si la donnée est absente, garantissant des messages d'erreur explicites plutôt qu'un `null` silencieux.

4.3. Cohérence REST / GraphQL

L'ensemble des endpoints REST du service Véhicules dispose d'un équivalent GraphQL, assurant une parité complète entre les deux interfaces. Trois opérations (`updateVehicle`, `deleteVehicle`, `vehicleAssignments`) ont été ajoutées au schéma initial pour combler les écarts identifiés.

Méthode	Endpoint REST	Équivalent GraphQL	Statut
POST	/vehicles	mutation `createVehicle`	**OK**
GET	/vehicles	query `vehicles`	**OK**
GET	/vehicles/ :id	query `vehicle`	**OK**
PUT	/vehicles/ :id	mutation `updateVehicle`	**OK**
PATCH	/vehicles/ :id/status	mutation `updateVehicleStatus`	**OK**
DELETE	/vehicles/ :id	mutation `deleteVehicle`	**OK**
POST	/vehicles/ :id/assignments	mutation `assignVehicle`	**OK**
GET	/vehicles/ :id/assignments	query `vehicleAssignments`	**OK**
DELETE	/vehicles/ :id/assignments/current	mutation `unassignVehicle`	**OK**

4.4. Gestion des stubs

Les résolveurs non encore branchés sont centralisés dans `stubResolvers.ts`. Deux comportements sont distingués :

— **Queries et Mutations** : lèvent une `GraphQLError` avec le code `NOT_IMPLEMENTED` et un message indiquant le champ concerné.

— **Subscriptions** : lèvent une erreur `SUBSCRIPTIONS_DISABLED`, signalant que le transport WebSocket (`graphql-ws`) n'est pas encore configuré sur le gateway.

Les champs concernés par les stubs couvrent les services à venir : `drivers`, `alerts`, `vehicleLocation`, `locationHistory` et `maintenanceRecords` pour les queries ; `createDriver`, `acknowledgeAlert`, `createMaintenanceRecord` et leurs variantes pour les mutations.

Cette approche **fail-fast** garantit qu'aucun appel à un service non implémenté ne passe silencieusement, facilitant le débogage lors des semaines suivantes.